Compte Rendu - Assistance Intelligence Artificielle (IA)
Atelier 5 : File d'Attente Offline-First avec Géolocalisation

Étudiant	Iheb Najjar
Classe	DSI 31
Matière	Développement Cross-Platform (Flutter)
Outil d'IA	Antigravity (Google DeepMind)

Introduction

Dans le cadre de l'Atelier 5, nous avons étendu l'application de salle d'attente (TP4) pour intégrer la persistance locale avec SQLite (`sqflite`), la capture de position GPS (`geolocator`), et la synchronisation non bloquante vers Supabase. L'intelligence artificielle a été sollicitée comme assistant technique pour surmonter des erreurs d'environnement de test, des dépréciations d'API et des problématiques d'architecture offline-first.

Les 5 Prompts Soumis à l'IA

Prompt 1 : Dépréciations et permissions de `geolocator`

> *Prompt : "Dans `GeolocationService`, `desiredAccuracy` et `timeLimit` sont marqués comme dépréciés dans la version 14.x de `geolocator`. Quelles sont les nouvelles classes recommandées, et comment configurer les permissions requises dans `AndroidManifest.xml` et `Info.plist` pour Android et iOS ?"*

Apport de l'IA : Proposition d'utiliser `locationSettings: const LocationSettings(accuracy: LocationAccuracy.low, timeLimit: Duration(seconds: 5))` et ajout des balises `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION` ainsi que la clé iOS `NSLocationWhenInUseUsageDescription`.

Prompt 2 : Isolation des bases de données SQLite en mémoire pendant les tests

> *Prompt : "Lors de l'exécution de `flutter test`, le deuxième test échoue avec un compte d'éléments inattendu car les données insérées dans le premier test restent présentes dans `:memory:`. Comment isoler les tests ou réinitialiser la table SQLite proprement ?"*

Apport de l'IA : Explication du comportement du cache partagé dans `sqflite_common_ffi` et implémentation d'une méthode `clear()` dans `LocalQueueService` appelée dans `setUp()` pour garantir que chaque test démarre avec une table vide.

Prompt 3 : Erreur `Unsupported operation: read-only` sur les résultats SQLite

> *Prompt : "Quand je tente de trier directement le résultat retourné par `db.query()`, Flutter lève une exception `Unsupported operation: read-only`. Pourquoi et comment y remédier ?"*

Apport de l'IA : Identification que `db.query()` retourne une `QueryResultSet` immuable. Solution : cloner la liste via `List<Map<String, dynamic>>.from(localList)` avant d'appliquer `.sort()`.

Prompt 4 : Architecture Offline-First et mise à jour optimiste de l'UI

> *Prompt : "Comment s'assurer que l'utilisateur voit son client immédiatement dans la file d'attente sans attendre la fin de la requête GPS ou l'insertion réseau Supabase ?"*

Apport de l'IA : Conception du pattern de mise à jour optimiste : ajout immédiat à la liste en mémoire avec notification des écouteurs (`notifyListeners()`), écriture asynchrone dans SQLite local, puis synchronisation distante non bloquante avec `unawaited(_syncLocalToRemote())`.

Prompt 5 : Support multi-plateforme, persistance Web (Edge) et redémarrage de l'application

> *Prompt : "Quand j'exécute l'application sur le navigateur Edge (`flutter run -d edge`), est-ce normal qu'après avoir ajouté des clients et fermé l'application, ils disparaissent au redémarrage ? Comment assurer une vraie persistance locale sur Web et Mobile sans driver C natif SQLite ?"*

Apport de l'IA : Explication de la différence entre SQLite natif (Android/iOS/Desktop) et l'environnement Web. Implémentation d'une persistance locale permanente dans `LocalQueueService` couplée à `shared_preferences` / `localStorage` pour le Web et réconciliation bidirectionnelle avec Supabase au démarrage, garantissant que les clients ne sont jamais perdus lors d'un redémarrage ou d'une déconnexion.

Bilan Personnel

L'utilisation de l'IA m'a permis de mieux comprendre le cycle de vie asynchrone en Flutter, l'isolation des tests unitaires et widgets avec `sqflite_common_ffi`, ainsi que les bonnes pratiques de synchronisation offline-first sans bloquer l'expérience utilisateur.